EXPERIMENT - 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed :

1. Hash Generation using SHA-256 :

o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce :

o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. **Proof-of-Work Puzzle Solving :**

o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction :

o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output :

Program 1 : Python program that uses the hashlib library to create the hash of a given string:
Code :

```
import hashlib
 #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
  #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
#error statement
```

Output :

```
Enter a string: Himanshi
Hash: 8086ec7f528f61709f87fe16d21cdf62b96f6998138395de65cef612ee26d68d
```

Program 2: Program to generate required target hash with input string and nonce

Code :

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce
  # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
 # Error
```

Output :

```
Enter a string: Himanshi
Enter the nonce: 15
Hash Code: 23fdec793cc828662dbecf77736e8ce3231c1c20755771e06832085c4f56a490
```

Program 3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

Code :

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
#Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
#Error
            print("Hash:",hash_code )
            return nonce
        nonce += 1
# Get user input
input_string = "A"
num_zeros = 1
# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)
# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
#Error
```

Output :

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

Program 4: Generating Merkle Tree for given set of Transactions
Code :

```
import hashlib
def build_merkle_tree(transactions):                                    # Error
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]
    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined =
hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(transactions[-1])                   #
Error
        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = ["Transaction 1", "Transaction 2", "Transaction 3",
"Transaction 4", "Transaction 5"]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)                                      # Error
```

Output :

```
Merkle Root: Transaction 5
```